

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

“JnanaSangama”, Belgaum -590014, Karnataka.



LAB REPORT
on

Machine Learning (23CS6PCMAL)

Submitted by

PRADHAN SAGAR K (1BM23CS237)

in partial fulfillment for the award of the degree of

BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE AND ENGINEERING



B.M.S. COLLEGE OF ENGINEERING
(Autonomous Institution under VTU)
BENGALURU-560019
Jan-2026 to May-2026

B.M.S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering



CERTIFICATE

This is to certify that the Lab work entitled “Machine Learning (23CS6PCMAL)” carried out by **Pradhan Sagar K (1BM23CS237)**, who is bonafide student of **B.M.S. College of Engineering**. It is in partial fulfillment for the award of **Bachelor of Engineering in Computer Science and Engineering** of the Visvesvaraya Technological University, Belgaum. The Lab report has been approved as it satisfies the academic requirements in respect of an Machine Learning (23CS6PCMAL) work prescribed for the said degree.

Dr. A. Sarkunavathi Assistant Professor Department of CSE, BMSCE	Dr. Kavitha Sooda Professor & HOD Department of CSE, BMSCE
--	--

Index

Sl. No.	Date	Experiment Title	Page No.
1	02-02-2026	Write a python program to import and export data using Pandas library functions	1
2	09-02-2026	Demonstrate various data pre-processing techniques for a given dataset	6
3	16-02-2026	Implement Linear and Multi-Linear Regression algorithm using appropriate dataset	17
4	23-02-2026	Build KNN Classification model for a given dataset.	24
5	02-03-2026	Use an appropriate data set for building the decision tree (ID3) and apply this knowledge to classify a new sample.	31
6	09-03-2026	Build Support vector machine model for a given dataset	39
7	16-03-2026	Implement Random forest ensemble method on a given dataset	45
8	23-03-2026	Implement Boosting ensemble method on a given dataset.	48
9	13-04-2026	Build k-Means algorithm to cluster a set of data stored in a .CSV file.	51
10	13-04-2026	Implement Dimensionality reduction using Principal Component Analysis (PCA) method.	54
11	13-04-2026	Build Logistic Regression Model for a given dataset	56

Github Link:

<https://github.com/pradhanSagarK-bmsce/6E-ML-LAB-1BM23CS237.git>

Program-1

Write a python program to import and export data using Pandas library functions

Observation :

Lab-0

ML algorithm calculation platform exploration

> miniconda set up complete

Tools learnt

- numpy - arrays
- Pandas - Dataset tables
- matplotlib - graphs

```
from sklearn.datasets import load_iris
import pandas as pd

iris = load_iris()
df = pd.DataFrame(iris.data, columns=iris.feature_names)
df['species'] = iris.target
df.head()
```

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	Species
0	5.1	3.5	1.4	0	
1	4.9	3.0	1.4	0	
2	4.7	3.2	1.3	0	
3	4.6	3.1	1.5	0	
4	5.0	3.6	1.4	0	

```

df.shape
(150, 5)

df.columns

df.info()

df.describe()

df['species'].value_counts()

species
0    50
1    50
2    50

Name: count, dtype: int64

Plotting graph

import matplotlib.pyplot as plt
plt.scatter(df['sepal length (cm)'], df['petal length (cm)'])

plt.xlabel('sepal length')
plt.ylabel('petal length')
plt.show()

```

```

df[df['species'] == 0].head()
df[df['species'] == 1].head()

df.corr()

df.sort_values(by='petal length (cm)').head()

```

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	species
22	4.6	3.6	1.0	0.2	0
13	4.3	3.0	1.1	0.1	0
14	5.8	4.0	1.2	0.2	0
35	5.0	3.2	1.2	0.2	0
16	5.4	3.9	1.3	0.4	0

Code :

```
from sklearn.datasets import load_iris
import pandas as pd
iris = load_iris()
df = pd.DataFrame(iris.data, columns=iris.feature_names)
df['species'] = iris.target
df.head()
```

sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	species
0	5.1	3.5	1.4	0.2
1	4.9	3.0	1.4	0.2
2	4.7	3.2	1.3	0.2
3	4.6	3.1	1.5	0.2
4	5.0	3.6	1.4	0.2

```
df.shape
```

(150, 5)

```
df.columns
```

```
Index(['sepal length (cm)', 'sepal width (cm)', 'petal length (cm)',
      'petal width (cm)', 'species'],
      dtype='object')
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 150 entries, 0 to 149
```

```
Data columns (total 5 columns):
```

#	Column	Non-Null Count	Dtype
---	-----	-----	-----
0	sepal length (cm)	150 non-null	float64
1	sepal width (cm)	150 non-null	float64
2	petal length (cm)	150 non-null	float64
3	petal width (cm)	150 non-null	float64
4	species	150 non-null	int64

```
dtypes: float64(4), int64(1)
```

```
memory usage: 6.0 KB
```

```
df.describe()
```

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	species
count	150.000000	150.000000	150.000000	150.000000	150.000000
mean	5.843333	3.057333	3.758000	1.199333	1.000000
std	0.828066	0.435866	1.765298	0.762238	0.819232
min	4.300000	2.000000	1.000000	0.100000	0.000000
25%	5.100000	2.800000	1.600000	0.300000	0.000000
50%	5.800000	3.000000	4.350000	1.300000	1.000000
75%	6.400000	3.300000	5.100000	1.800000	2.000000

max	7.900000	4.400000	6.900000	2.500000	2.000000
------------	----------	----------	----------	----------	----------

```
df['species'].value_counts()
```

```
species
```

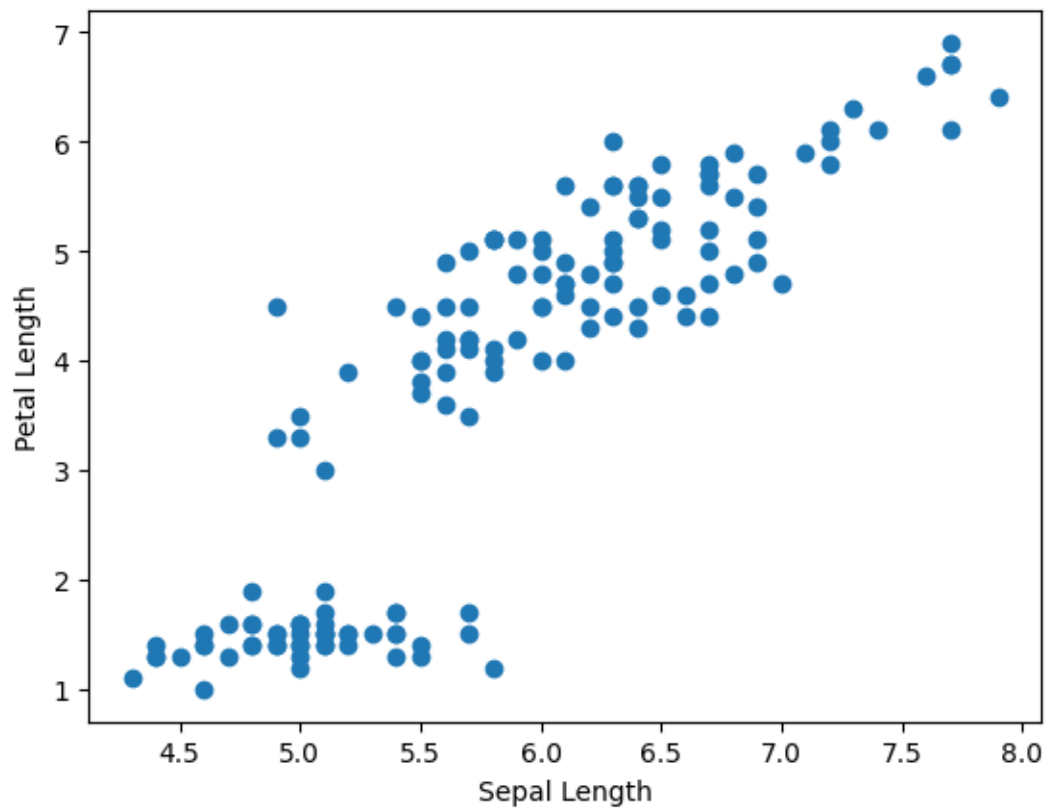
```
0    50
```

```
1    50
```

```
2    50
```

```
Name: count, dtype: int64
```

```
import matplotlib.pyplot as plt
plt.scatter(df['sepal length (cm)'], df['petal length (cm)'])
plt.xlabel('Sepal Length')
plt.ylabel('Petal Length')
plt.show()
```



```
df[df['species'] == 0].head()
df[df['species'] == 1].head()
```

sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	species
50	7.0	3.2	4.7	1.4
51	6.4	3.2	4.5	1.5
52	6.9	3.1	4.9	1.5
53	5.5	2.3	4.0	1.3
54	6.5	2.8	4.6	1.5

```
df.corr()
```

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	species
sepal length (cm)	1.000000	-0.117570	0.871754	0.817941	0.782561
sepal width (cm)	-0.117570	1.000000	-0.428440	-0.366126	-0.426658
petal length (cm)	0.871754	-0.428440	1.000000	0.962865	0.949035
petal width (cm)	0.817941	-0.366126	0.962865	1.000000	0.956547
species	0.782561	-0.426658	0.949035	0.956547	1.000000
df.sort_values(by='petal length (cm)').head()					

sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	species
22	4.6	3.6	1.0	0.2
13	4.3	3.0	1.1	0.1
14	5.8	4.0	1.2	0.2
35	5.0	3.2	1.2	0.2
16	5.4	3.9	1.3	0.4

Program-2

Demonstrate various data pre-processing techniques for a given dataset

Observation :

Lab -1 9/12/26

End to End Machine Learning Lab
Application: House Price Prediction

Step 0: Import Required Libraries

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
```

Step 1: Look at the Big Picture

Problem Type: Supervised Learning - Regression
Objective: Predict median house value based on housing features
Evaluation metrics: RMSE, MAE, R^2

Step 2: Get the Data (3)

```
# load california housing dataset
housing = fetch_california_housing(as_frame=True)
# create Dataframe
df = housing.frame
# Display basic info
df.head()
df.shape
df.info()
```

clean 'median' column from 'DataFrame'
RangeIndex: 20640 entries, 0 to 20639
Data columns (total 9 columns):

#	Column	Non-Null Count	Dtype
0	MedInc	20640 non-null	float64
1	HouseAge	20640 non-null	float64
2	AvgRooms	20640 non-null	float64
3	AvgBedrms	20640 non-null	float64
4	population	20640 non-null	float64
5	AveOccup	20640 non-null	float64
6	Latitude	20640 non-null	float64
7	Longitude	20640 non-null	float64
8	MedHouseVal	20640 non-null	float64

dtypes: float64 (9)
memory usage: 1.4 MB

```

df.describe()
# check missing values
df.isnull().sum()

Median 0
House Age 0
Area Room 0
Price Bedroom 0
Population 0
Avg Occup 0
Latitude 0
Longitude 0
MedianHouseVal 0

dtype: int64

Step 4: Visualize the Data
4.1 Distribution of House Prices
plt.figure()
sns.histplot(df['MedianHouseVal'], bins=30, kde=True)
plt.xlabel('Median House Value')
plt.ylabel('Frequency')
plt.title('Distribution of House Prices')
plt.show()

4.2 Income vs House Price
plt.figure()
sns.scatterplot(x=df['MedianInc'], y=df['MedianHouseVal'], alpha=0.8)
plt.xlabel('Median Income')
plt.ylabel('Median House Value')
plt.title('Income vs House Price')
plt.show()

```

```

Step 6: Select and Train a Baseline Model
6.1 Linear Regression
lin_reg = LinearRegression()
lin_reg.fit(X_train_scaled, y_train)
y_pred_lr = lin_reg.predict(X_test_scaled)
mse_lr = np.sqrt(mean_squared_error(y_test, y_pred_lr))
rmse_lr = mse_lr, 2*lr

Coefficients: (0.744954028203491, 0.533013939645134, 0.5765013224026747)

Step 7: Train Advanced Models
7.1 Decision Tree Regression
dt = DecisionTreeRegressor(random_state=42)
dt.fit(X_train, y_train)
y_pred_dt = dt.predict(X_test)
rmse_dt = np.sqrt(mean_squared_error(y_test, y_pred_dt))
rmse_dt

np.float64(0.713837737394808)

7.2 Random Forest Regressor
rf = RandomForestRegressor(random_state=42)
rf.fit(X_train, y_train)
y_pred_rf = rf.predict(X_test)
rmse_rf = np.sqrt(mean_squared_error(y_test, y_pred_rf))
rmse_rf

np.float64(0.5132629837109113)

Step 8: Fine Tune the Model
8.1 Hyperparameter Tuning (Random Forest)
param_grid = {
    'n_estimators': [50, 500],
    'max_depth': [None, 10, 20],
    'min_samples_split': [2, 5]
}
grid_search = GridSearchCV(
    rf, param_grid, cv=5,
    scoring='neg-mean-squared-error'
)
grid_search.fit(X_train, y_train)
best_model = grid_search.best_estimator_
y_pred_best = best_model.predict(X_test)
rmse_best = np.sqrt(mean_squared_error(y_test, y_pred_best))
rmse_best

0.5132629837109113

Step 9: Model Comparison
result_dict = {'model': ['Linear Regression', 'DecisionTree',
    'RandomForest', 'TunedRandom Forest'],
    'RMSE': [rmse_lr, rmse_dt, rmse_rf, rmse_best]}

3
result = pd.DataFrame(
    result_dict
)

result

```

Step 10: Best performing model

(10)

```
min_row = results.loc[results['rmse'].idxmin()]\nprint(f"Best performing model: {min_row['model']}\n\nwith best RMSE: {min_row['rmse']}")
```

Best performing model: Random Forest

With best RMSE: 0.5128644706054187

Which model performed best?

Random Forest performed best as its RMSE is lesser

Which did Random Forest outperform Linear Regression?

Because, Linear regression assumes linear relationship among data which is not true everytime

How did tuning improve performance?

In this case tuning doesn't improve performance much as it also depends on tuned features

What additional data could further improve predictions?

may be nearby famous place and distance of the may require further improved predictions

Code:

Step 0: Import Required Libraries

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
```

Step 1: Look at the Big Picture

Problem Type: Supervised Learning – Regression.

Objective: Predict median house value based on housing features.

Evaluation Metrics: RMSE, MAE, R^2 .

Step 2: Get the Data

```
# Load California housing dataset
housing = fetch_california_housing(as_frame=True)
# Create DataFrame
df = housing.frame
# Display basic info
df.head()
df.shape
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 20640 entries, 0 to 20639
Data columns (total 9 columns):
#   Column          Non-Null Count  Dtype
---  -
0   MedInc           20640 non-null  float64
1   HouseAge         20640 non-null  float64
2   AveRooms         20640 non-null  float64
3   AveBedrms        20640 non-null  float64
4   Population       20640 non-null  float64
5   AveOccup         20640 non-null  float64
6   Latitude         20640 non-null  float64
7   Longitude        20640 non-null  float64
8   MedHouseVal      20640 non-null  float64
dtypes: float64(9)
```

memory usage: 1.4 MB

Step 3: Discover the Data (EDA)

```
df.describe()
# Check missing values
df.isnull().sum()
```

```
MedInc          0
HouseAge        0
AveRooms        0
AveBedrms       0
Population      0
AveOccup        0
Latitude        0
Longitude       0
MedHouseVal     0
dtype: int64
```

Step 4: Visualize the Data

4.1 Distribution of House Prices

```
plt.figure()
sns.histplot(df['MedHouseVal'], bins=30, kde=True)
plt.xlabel('Median House Value')
plt.ylabel('Frequency')
plt.title('Distribution of House Prices')
plt.show()
```



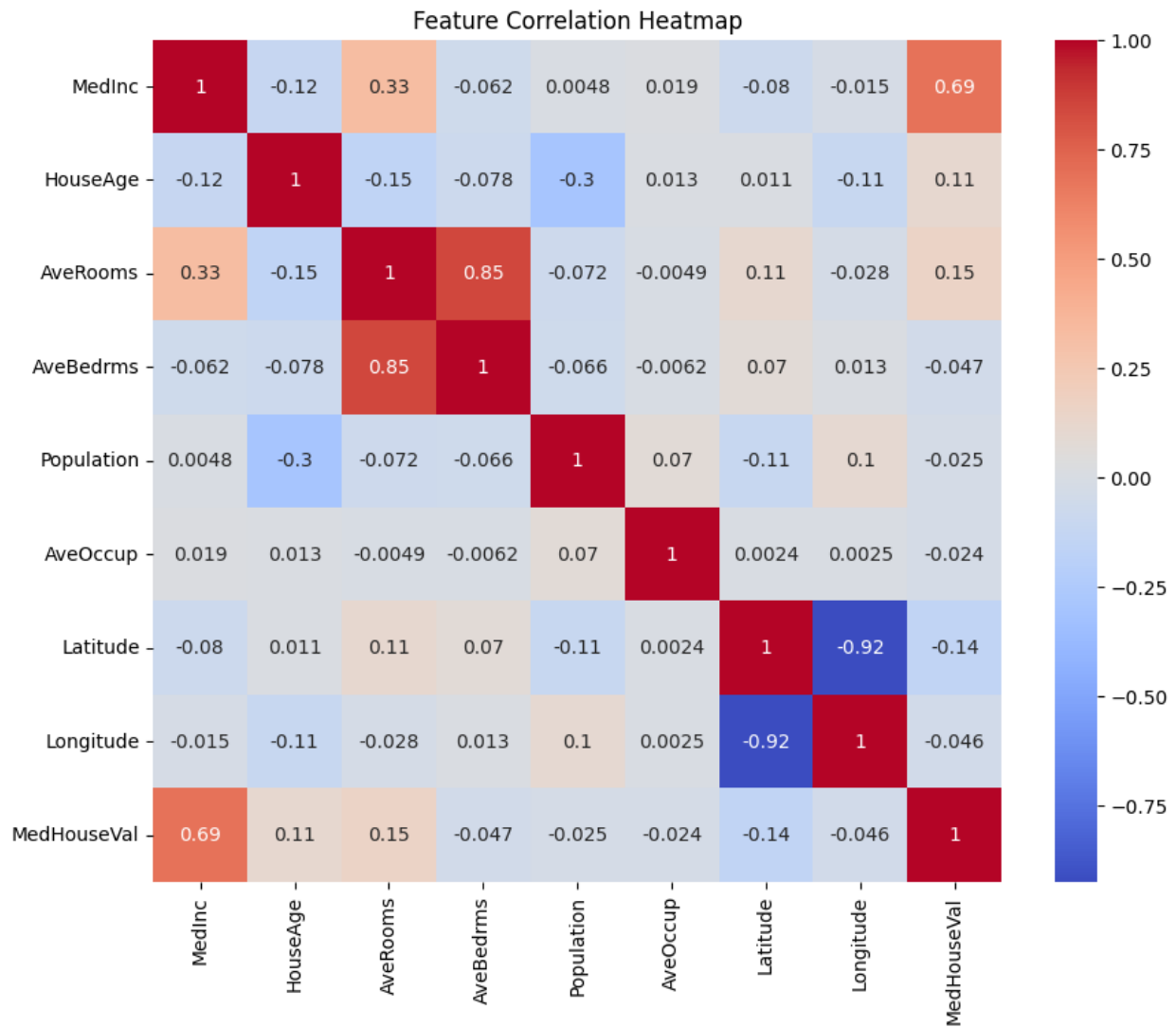
4.2 Income vs House Price

```
plt.figure()
sns.scatterplot(x=df['MedInc'], y=df['MedHouseVal'], alpha=0.5)
plt.xlabel('Median Income')
plt.ylabel('Median House Value')
plt.title('Income vs House Price')
plt.show()
```



4.3 Correlation Heatmap

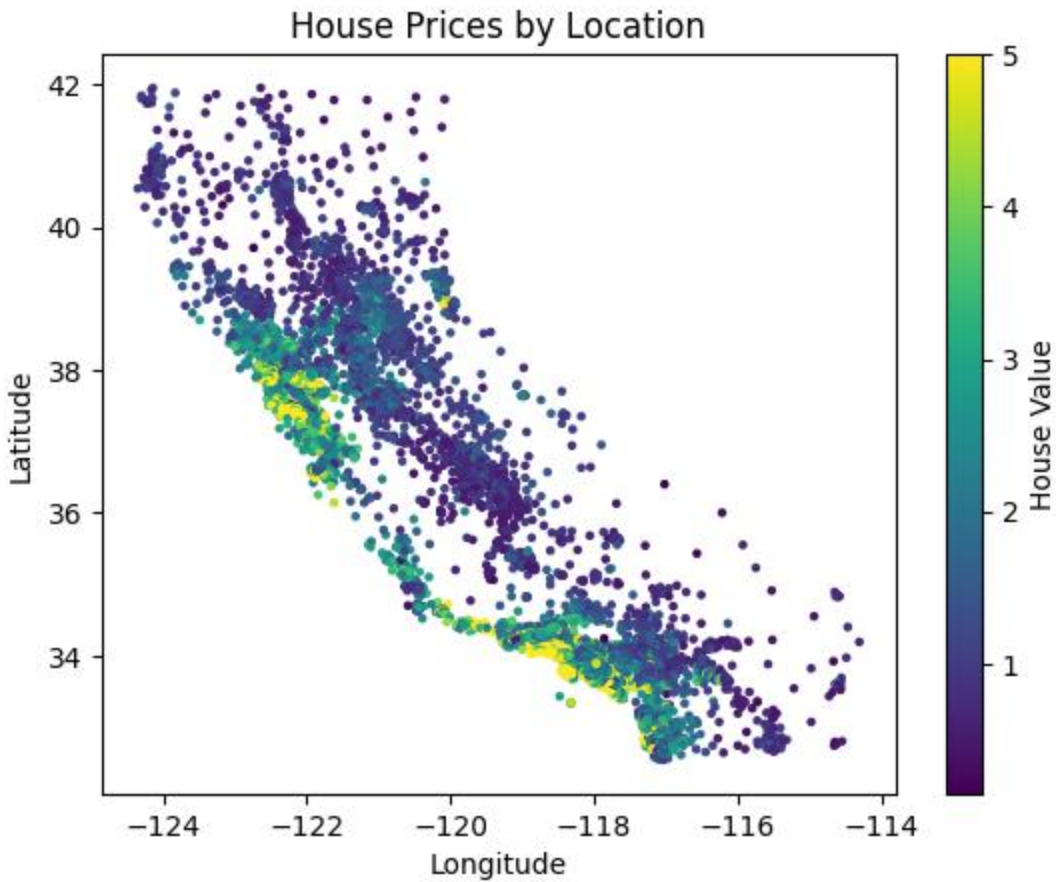
```
plt.figure(figsize=(10,8))
sns.heatmap(df.corr(), annot=True, cmap='coolwarm')
plt.title('Feature Correlation Heatmap')
plt.show()
```



4.4 Location-based Visualization

```
plt.figure()
plt.scatter(df['Longitude'], df['Latitude'],
c=df['MedHouseVal'], cmap='viridis', s=5)

plt.colorbar(label='House Value')
plt.xlabel('Longitude')
plt.ylabel('Latitude')
plt.title('House Prices by Location')
plt.show()
```



Step 5: Prepare the Data

5.1 Feature Engineering

```
df['RoomsPerHousehold'] = df['AveRooms'] / df['HouseAge']
```

5.2 Train-Test Split

```
X = df.drop('MedHouseVal', axis=1)
y = df['MedHouseVal']
X_train, X_test, y_train, y_test = train_test_split(
X, y, test_size=0.2, random_state=42
)
```

5.3 Feature Scaling

```
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

Step 6: Select and Train a Baseline Model

6.1 Linear Regression

```
lin_reg = LinearRegression()
lin_reg.fit(X_train_scaled, y_train)
y_pred_lr = lin_reg.predict(X_test_scaled)
rmse_lr = np.sqrt(mean_squared_error(y_test, y_pred_lr))
mae_lr = mean_absolute_error(y_test, y_pred_lr)
r2_lr = r2_score(y_test, y_pred_lr)
rmse_lr, mae_lr, r2_lr
```

(np.float64(0.7449540928043491), 0.5330139776645934, 0.5765012224420747)

Step 7: Train Advanced Models

7.1 Decision Tree Regressor

```
dt = DecisionTreeRegressor(random_state=42)
dt.fit(X_train, y_train)

y_pred_dt = dt.predict(X_test)
rmse_dt = np.sqrt(mean_squared_error(y_test, y_pred_dt))
rmse_dt
```

np.float64(0.7138377373947808)

7.2 Random Forest Regressor

```
rf = RandomForestRegressor(random_state=42)
rf.fit(X_train, y_train)
y_pred_rf = rf.predict(X_test)
rmse_rf = np.sqrt(mean_squared_error(y_test, y_pred_rf))
rmse_rf
```

np.float64(0.5128644706054181)

Step 8: Fine-Tune the Model

8.1 Hyperparameter Tuning (Random Forest)

```
param_grid = {
    'n_estimators': [50, 100],
    'max_depth': [None, 10, 20],
    'min_samples_split': [2, 5]
}
grid_search = GridSearchCV(
    rf, param_grid, cv=3,
    scoring='neg_mean_squared_error'
)
grid_search.fit(X_train, y_train)
best_model = grid_search.best_estimator_
```

```

y_pred_best = best_model.predict(X_test)
rmse_best = np.sqrt(mean_squared_error(y_test, y_pred_best))
rmse_best

```

```
np.float64(0.5132629837109113)
```

Step 9: Model Comparison

```

results_data = {'Model': ['Linear Regression', 'Decision Tree', 'Random Forest', 'Tuned
Random Forest'],
'RMSE': [rmse_lr, rmse_dt, rmse_rf, rmse_best]
}
results = pd.DataFrame(
    results_data
)

results

```

	Model	RMSE
0	Linear Regression	0.744954
1	Decision Tree	0.713838
2	Random Forest	0.512864
3	Tuned Random Forest	0.513263

Best performing Model

```

min_row = results.loc[results['RMSE'].idxmin()]

print(f"Best performing model : {min_row['Model']} \nWith best RMSE :
{min_row['RMSE']}")

```

```

Best performing model : Random Forest
With best RMSE : 0.5128644706054181

```

Program-3

Implement Linear and Multi-Linear Regression algorithm using appropriate dataset

Observation :

Lab-2 16-2-26

Implement ~~Linear~~ Single & multi-variable
Linear Regression using appropriate dataset

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
from mpl_toolkits.mplot3d import Axes3D
```

Single variable Linear Regression

Step 1 : Get Data

```
X_study_hours = np.array([
    1, 2, 3, 3.5, 4, 4.5, 5, 5.5, 6, 6.5, 7, 7.5, 8, 8.5, 9, 9.5,
    10, 10.5, 11, 11.5, 12, 12.5, 13, 13.5, 14, 15, 16, 17, 18]).reshape(-1, 1)
Y_exam_scores = np.array([35, 44, 50, 48, 58, 62, 67, 70, 68,
    75, 78, 82, 80, 88, 90, 92, 89, 95, 97, 93, 100, 102,
    99, 105, 107, 104, 110, 108, 112, 109])
```

Step-2 Perform Regression

```
lin_reg = LinearRegression()
lin_reg.fit(X_study_hours, Y_exam_scores)
Y_pred = lin_reg.predict(X_study_hours)

plt.scatter(X_study_hours, Y_exam_scores, label="Data points")
plt.plot(X_study_hours, Y_pred, color='red', label="regression line")
plt.ylabel("Exam Scores")
plt.xlabel("Study hours")
plt.legend()
plt.title("Study hours vs Exam Score")
```

```

mse = mean_squared_error(Y_exam_scores, y_pred)
rmse = np.sqrt(mse)
print(f"Root Mean Squared Error (RMSE): {rmse:.2f}")
print(f"R^2 score: {r2:.2f}")

hours = np.array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
predicted_score = lin_reg.predict(hours)
print(f"Predicted exam score for {hours[0]} hours: {predicted_score[0]:.2f}")

Output
Root Mean Squared Error (RMSE): 6.30
R^2 Score: 0.91
Predict exam score for 9 hours: 84.34

Multi-variable Linear Regression
X = np.array([
    [1, 5],
    [2, 6],
    [3, 5],
    [4, 7],
    [5, 6],
    [6, 8],
    [7, 7],
    [8, 8] ])

Y = np.array([35, 40, 50, 55, 65, 70, 75, 80])

lin_reg = LinearRegression()
lin_reg.fit(X, Y)

y_pred = lin_reg.predict(X)

mse = mean_squared_error(Y, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(Y, y_pred)
print(f"Root mean Squared Error (RMSE): {rmse:.2f}")
print(f"R^2 score: {r2:.2f}")

new_data = np.array([9, 7])
predicted_score = lin_reg.predict(new_data)
print(f"Predicted exam score for {new_data[0]} hours: {predicted_score[0]:.2f}")

fig = plt.figure()
ax = fig.add_subplot(111, projection='3d')
ax.scatter(X[:, 0], X[:, 1], Y, color='blue', label='data point')
ax.set_xlabel("Study hours")
ax.set_ylabel("Sleep hours")
ax.set_zlabel("Exam scores")
ax.set_title("Study hours & sleep hours vs Exam score")

```

```

Study_range = np.linspace(X[:, 0].min(), X[:, 0].max(), 10)
sleep_range = np.linspace(X[:, 1].min(), X[:, 1].max(), 10)

study_grid, sleep_grid = np.meshgrid(Study_range, sleep_range)

pred_grid = lin_reg.predict(np.c_[study_grid.ravel(), sleep_grid.ravel()]).reshape(study_grid.shape)

ax.plot_surface(study_grid, sleep_grid, pred_grid, color='red', alpha=0.5, label='Regression Plane')

plt.show()

Output
Root Mean Squared Error (RMSE): 1.47
R^2 Score: 0.99
Predicted exam score for 9 study hours and 7 sleep hours: 90.16

```

Code :

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
from mpl_toolkits.mplot3d import Axes3D
```

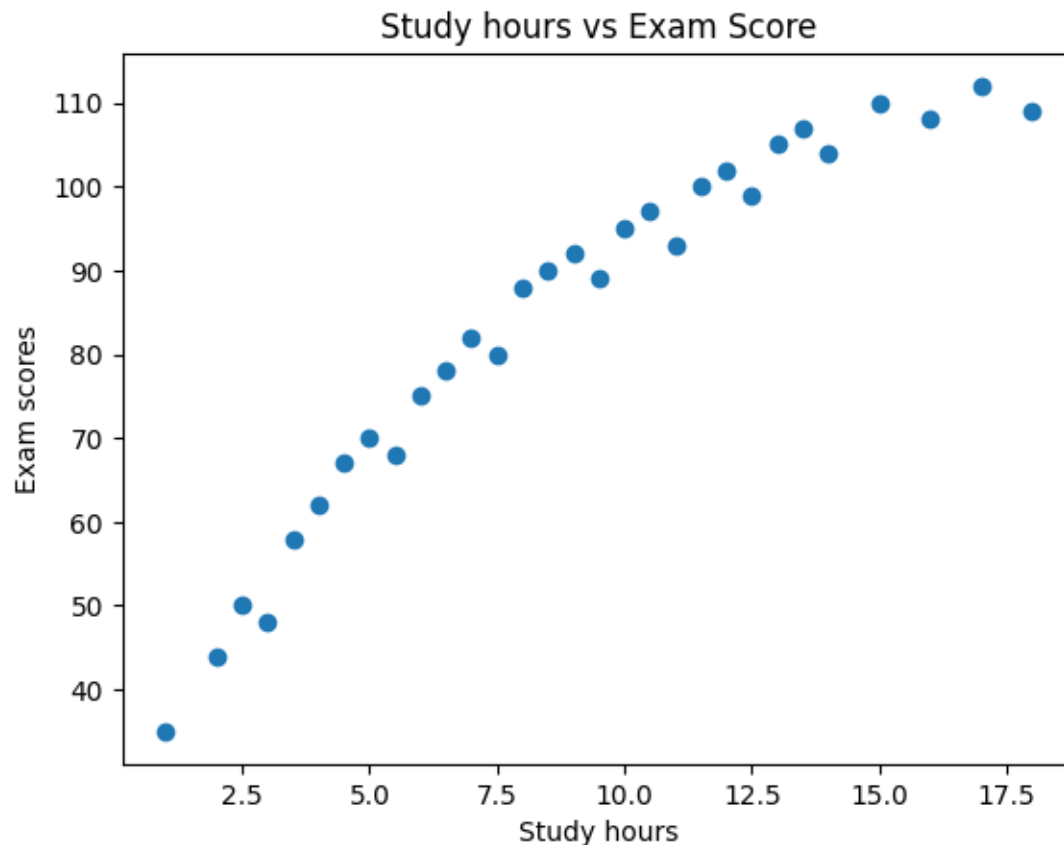
Single variable Linear Regression

Step - 1 Get Data

```
X_study_hours = np.array([
    1, 2, 2.5, 3, 3.5, 4, 4.5, 5, 5.5, 6,
    6.5, 7, 7.5, 8, 8.5, 9, 9.5, 10, 10.5, 11,
    11.5, 12, 12.5, 13, 13.5, 14, 15, 16, 17, 18
]).reshape(-1,1)

Y_exam_scores = np.array([
    35, 44, 50, 48, 58, 62, 67, 70, 68, 75,
    78, 82, 80, 88, 90, 92, 89, 95, 97, 93,
    100, 102, 99, 105, 107, 104, 110, 108, 112, 109
])

plt.scatter(X_study_hours,Y_exam_scores)
plt.ylabel("Exam scores")
plt.xlabel("Study hours")
plt.title("Study hours vs Exam Score")
plt.show()
```



Step - 2 Perform Regression

```
lin_reg = LinearRegression()

lin_reg.fit(X_study_hours,Y_exam_scores)

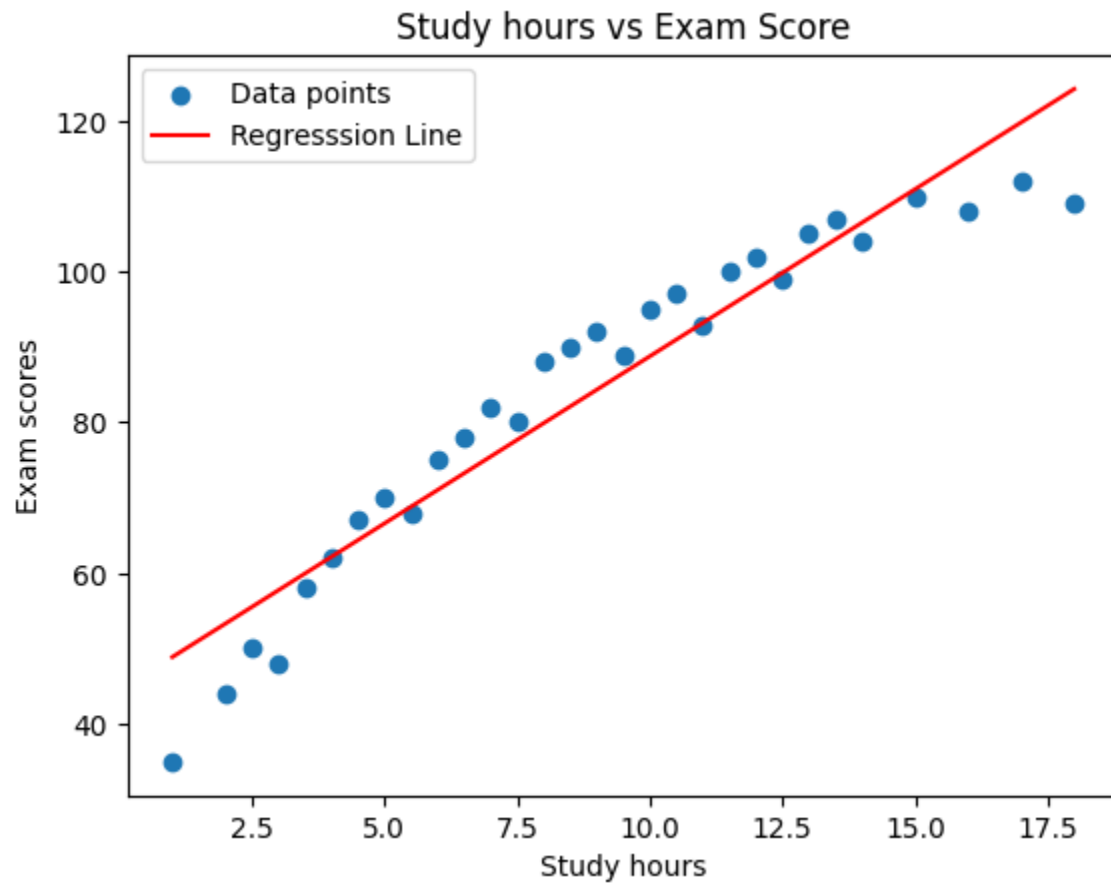
y_pred = lin_reg.predict(X_study_hours)

plt.scatter(X_study_hours,Y_exam_scores,label="Data points")
plt.plot(X_study_hours,y_pred,color='red',label="Regression Line")
plt.ylabel("Exam scores")
plt.xlabel("Study hours")
plt.legend()
plt.title("Study hours vs Exam Score")
plt.show()

mse = mean_squared_error(Y_exam_scores, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(Y_exam_scores, y_pred)
print(f"Root Mean Squared Error (RMSE): {rmse:.2f}")
print(f"R^2 Score: {r2:.2f}")

hours = np.array([[9]]) # 9 hours of study
predicted_score = lin_reg.predict(hours)
```

```
print(f"Predicted exam score for {hours[0][0]} hours: {predicted_score[0]:.2f}")
```



Root Mean Squared Error (RMSE): 6.30

R² Score: 0.91

Predicted exam score for 9 hours: 84.34

Multivariable Linear Regression

```
# [Study hours, Sleep hours]
X = np.array([
    [1, 5],
    [2, 6],
    [3, 5],
    [4, 7],
    [5, 6],
    [6, 8],
    [7, 7],
    [8, 8]
])

# Exam scores
Y = np.array([35, 40, 50, 55, 65, 70, 75, 80])
```

```

lin_reg = LinearRegression()
lin_reg.fit(X, Y)

y_pred = lin_reg.predict(X)

mse = mean_squared_error(Y, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(Y, y_pred)
print(f"Root Mean Squared Error (RMSE): {rmse:.2f}")
print(f"R^2 Score: {r2:.2f}")

# Predict a new value
new_data = np.array([[9, 7]]) # 9 study hours, 7 sleep hours
predicted_score = lin_reg.predict(new_data)
print(f"Predicted exam score for {new_data[0][0]} study hours and {new_data[0][1]} sleep hours: {predicted_score[0]:.2f}")

fig = plt.figure()
ax = fig.add_subplot(111, projection='3d')
ax.scatter(X[:,0], X[:,1], Y, color='blue', label="Data points")
ax.set_xlabel("Study hours")
ax.set_ylabel("Sleep hours")
ax.set_zlabel("Exam scores")
ax.set_title("Study hours & Sleep hours vs Exam Score")

# Create grid for plane
study_range = np.linspace(X[:,0].min(), X[:,0].max(), 10)
sleep_range = np.linspace(X[:,1].min(), X[:,1].max(), 10)
study_grid, sleep_grid = np.meshgrid(study_range, sleep_range)
pred_grid = lin_reg.predict(np.c_[study_grid.ravel(),
sleep_grid.ravel()]).reshape(study_grid.shape)

# Plot regression plane
ax.plot_surface(study_grid, sleep_grid, pred_grid, color='red', alpha=0.5,
label="Regression Plane")
plt.show()

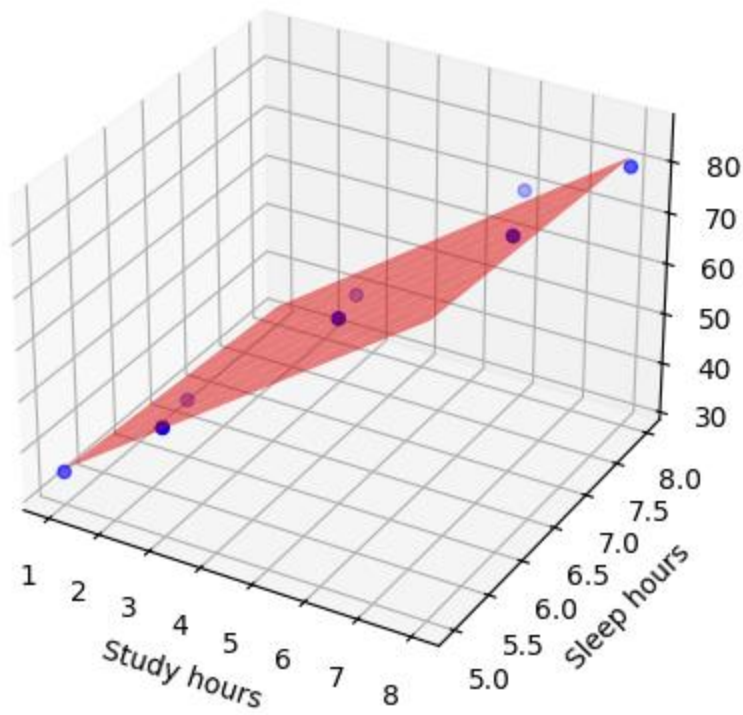
```

Root Mean Squared Error (RMSE): 1.47

R^2 Score: 0.99

Predicted exam score for 9 study hours and 7 sleep hours: 90.16

Study hours & Sleep hours vs Exam Score



Program-4

Build KNN Classification model for a given dataset.

Observation :

Lab-3 (X) 23-2-26

Implement KNN Classification and Regression

```
import numpy as np
import matplotlib.pyplot as plt
from collections import Counter
from sklearn.datasets import load_iris
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, mean_squared_error
from sklearn.preprocessing import StandardScaler
import math
```

KNN Classifier

```
class KNNClassifier:
    def __init__(self, K=3):
        self.k = K

    def fit(self, X, y):
        self.X_train = np.array(X)
        self.y_train = np.array(y)

    def predict(self, X):
        X = np.array(X)
        predictions = [self._predict(x) for x in X]
        return np.array(predictions)
```

```

def _predict(self, X):
    distance = np.linalg.norm(self.x_train - X, axis=1)
    k_indices = np.argsort(distance)[0:k]
    k_nearest_values = self.y_train[k_indices]
    return np.mean(k_nearest_values)

# KNN Regression
class KNNRegressor:
    def __init__(self, k=3):
        self.k = k

    def fit(self, x, y):
        self.x_train = np.array(x)
        self.y_train = np.array(y)

    def predict(self, x):
        x = np.array(x)
        predictions = [self._predict(x) for x in x]
        return np.array(predictions)

    def _predict(self, x):
        distance = np.linalg.norm(self.x_train - x, axis=1)
        k_indices = np.argsort(distance)[0:k]
        k_nearest_values = self.y_train[k_indices]
        return np.mean(k_nearest_values)

# Load dataset
iris = load_iris()
X = iris.data
y = iris.target

scaler = StandardScaler()
X = scaler.fit_transform(X)

x_train, x_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

clf = KNNClassifier(k=5)
clf.fit(x_train, y_train)

predictions = clf.predict(x_test)
print("Accuracy", accuracy_score(y_test, predictions))

output:
Accuracy: 1.0

```

```

# Regression
url = "https://github.com/jcason-data/maths/mpg.csv"
data = pd.read_csv(url)

data = data[['horsepower', 'mpg']].dropna()

X = data[['horsepower']].values
y = data[['mpg']].values

scaler = StandardScaler()
X = scaler.fit_transform(X)

x_train, x_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

reg = KNNRegressor(k=8)
reg.fit(x_train, y_train)

predictions = reg.predict(x_test)

mse = mean_squared_error(y_test, predictions)
rmse = math.sqrt(mse)

print("MSE", mse)
print("RMSE", rmse)

output:
MSE: 18.724
RMSE: 4.3271

```

Code :

KNN - Classification and Regression

```
import numpy as np
import matplotlib.pyplot as plt
from collections import Counter
from sklearn.datasets import load_iris
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, mean_squared_error
from sklearn.preprocessing import StandardScaler
import math

# KNN CLASSIFIER
class KNNClassifier:

    def __init__(self, k=3):
        self.k = k

    def fit(self, X, y):
        self.X_train = np.array(X)
        self.y_train = np.array(y)

    def predict(self, X):
        X = np.array(X)
        predictions = [self._predict(x) for x in X]
        return np.array(predictions)

    def _predict(self, x):

        distances = np.linalg.norm(self.X_train - x, axis=1)

        k_indices = np.argsort(distances)[:self.k]
        k_nearest_labels = self.y_train[k_indices]

        most_common = Counter(k_nearest_labels).most_common(1)
        return most_common[0][0]

# KNN REGRESSOR

class KNNRegressor:

    def __init__(self, k=3):
        self.k = k

    def fit(self, X, y):
        self.X_train = np.array(X)
        self.y_train = np.array(y)
```

```

def predict(self, X):
    X = np.array(X)
    predictions = [self._predict(x) for x in X]
    return np.array(predictions)

def _predict(self, x):
    distances = np.linalg.norm(self.X_train - x, axis=1)

    k_indices = np.argsort(distances)[:self.k]
    k_nearest_values = self.y_train[k_indices]

    return np.mean(k_nearest_values)

# CLASSIFICATION EXAMPLE

# Load dataset
iris = load_iris()
X = iris.data
y = iris.target

scaler = StandardScaler()
X = scaler.fit_transform(X)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

clf = KNNClassifier(k=5)
clf.fit(X_train, y_train)

predictions = clf.predict(X_test)
print("====KNN Classification====")
print("Accuracy:", accuracy_score(y_test, predictions))

# Use only first 2 features for visualization
X_vis = iris.data[:, :2]
y = iris.target

scaler = StandardScaler()
X_vis = scaler.fit_transform(X_vis)
X_train, X_test, y_train, y_test = train_test_split(
    X_vis, y, test_size=0.2, random_state=42
)

```

```

clf.fit(X_train, y_train)

plt.figure(figsize=(8, 6))

x_min, x_max = X_vis[:, 0].min() - 1, X_vis[:, 0].max() + 1
y_min, y_max = X_vis[:, 1].min() - 1, X_vis[:, 1].max() + 1
xx, yy = np.meshgrid(
    np.arange(x_min, x_max, 0.05),
    np.arange(y_min, y_max, 0.05)
)

grid = np.c_[xx.ravel(), yy.ravel()]
Z = clf.predict(grid)
Z = Z.reshape(xx.shape)

plt.contourf(xx, yy, Z, alpha=0.3)
plt.scatter(X_test[:, 0], X_test[:, 1], c=y_test, edgecolor='k')
plt.title("KNN Decision Boundary - Iris Dataset")
plt.xlabel("Sepal Length")
plt.ylabel("Sepal Width")
plt.show()

# Load Auto MPG dataset directly from UCI
url = "https://raw.githubusercontent.com/mwaskom/seaborn-data/master/mpg.csv"
data = pd.read_csv(url)

# Keep only horsepower and mpg
data = data[['horsepower', 'mpg']].dropna()

X = data[['horsepower']].values
y = data['mpg'].values

scaler = StandardScaler()
X = scaler.fit_transform(X)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

reg = KNNRegressor(k=8)
reg.fit(X_train, y_train)

predictions = reg.predict(X_test)

```

```

mse = mean_squared_error(y_test, predictions)
rmse = math.sqrt(mse)

print("=====KNN Regression=====")
print("MSE:", mse)
print("RMSE:", rmse)

sorted_idx = np.argsort(X_test[:, 0])
X_sorted = X_test[sorted_idx]
y_sorted = y_test[sorted_idx]
pred_sorted = predictions[sorted_idx]

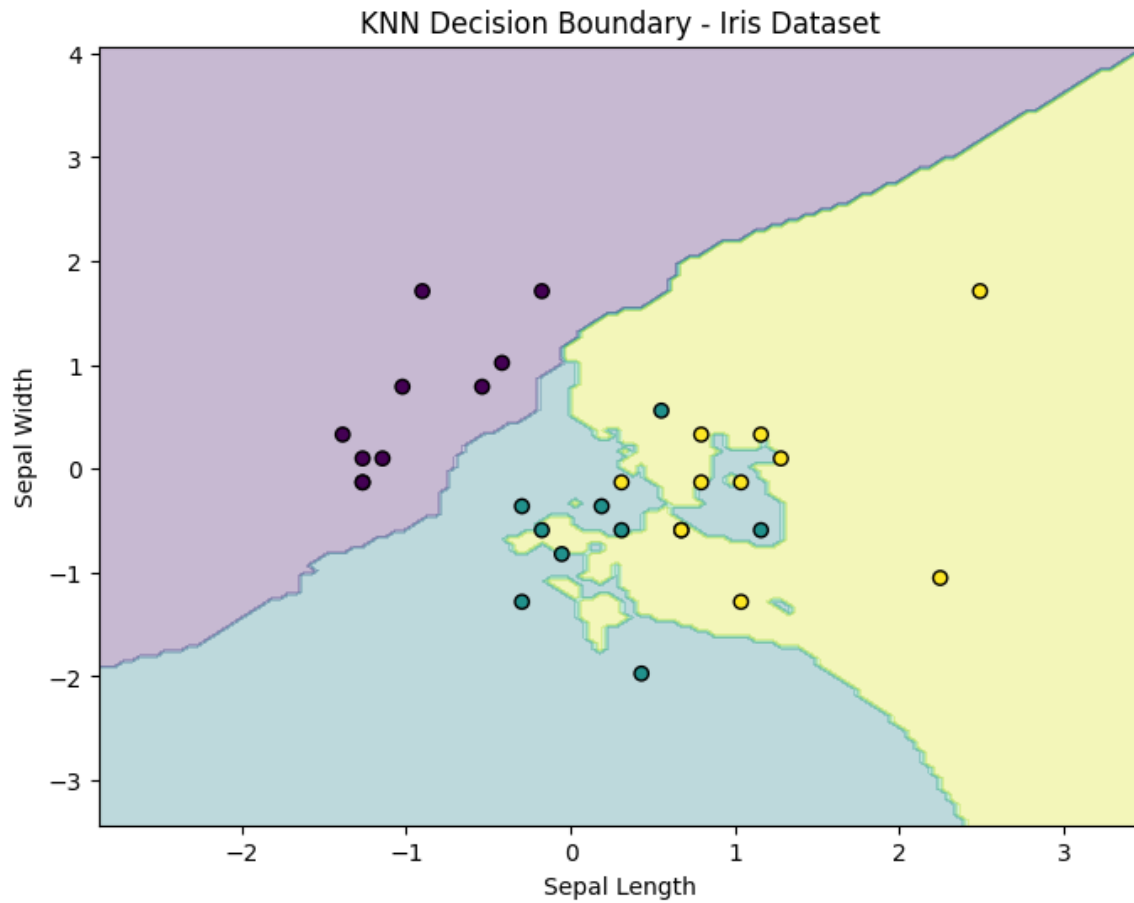
plt.figure(figsize=(10, 6))

plt.scatter(X_sorted, y_sorted, color='blue', alpha=0.6, label="Actual MPG")
plt.plot(X_sorted, pred_sorted, color='red', linewidth=2.5, label="KNN Prediction")

plt.title("KNN Regression - Auto MPG Dataset")
plt.xlabel("Horsepower (scaled)")
plt.ylabel("Miles Per Gallon (MPG)")
plt.legend()
plt.grid(True)
plt.show()

=====KNN Classification=====
Accuracy: 1.0

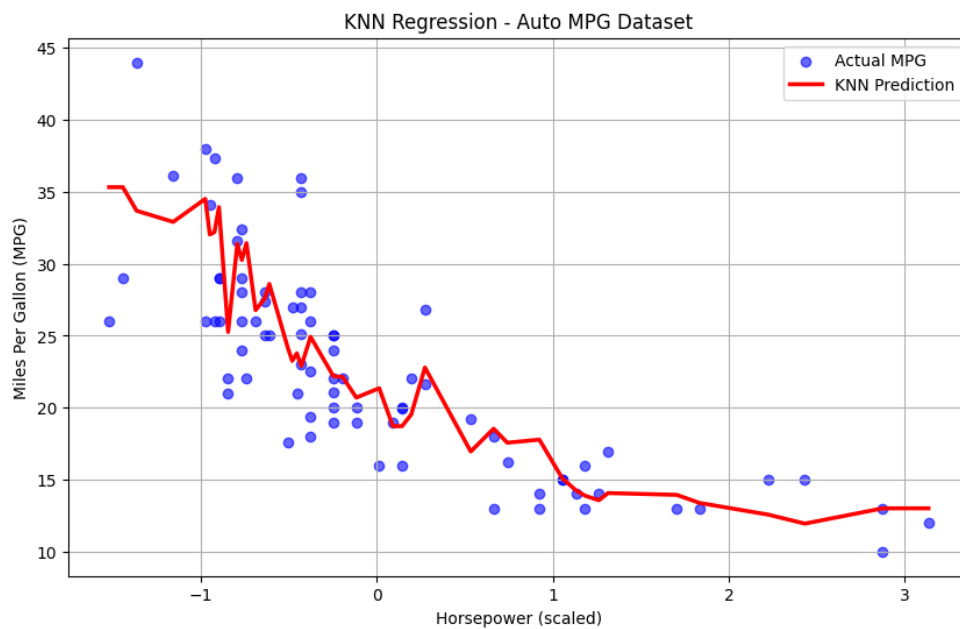
```



=====KNN Regression=====

MSE: 18.72448971518988

RMSE: 4.327180342346489



Program-5

Use an appropriate data set for building the decision tree (ID3) and apply this knowledge to classify a new sample.

Observation :

Lab - 4 2-3-26

Decision tree Classification using ID3
and Regression using CART

```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
from sklearn.datasets import load_iris, load_digits, load_svhn
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, mean_squared_error

# Classification Dataset 1 (Iris)
iris = load_iris()
X = iris.data[:, :2]
y = iris.target

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
                                                    random_state=42)

clf1 = DecisionTreeClassifier(max_depth=4)
clf1.fit(X_train, y_train)
pred1 = clf1.predict(X_test)
accuracy1 = accuracy_score(y_test, pred1)
print("Decision Tree Classification (Iris) Accuracy : ", accuracy1)
```

```

# Classifier Dataset 2 (Digits)
digits = load_digits()
X = digits.data
y = digits.target

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)

X_train, X_test, y_train, y_test = train_test_split(X_pca,
                                                    y, test_size=0.2, random_state=42)
clf = DecisionTreeClassifier(max_depth=6)
clf.fit(X_train, y_train)
pred2 = clf.predict(X_test)
accuracy2 = accuracy_score(y_test, pred2)
print("Decision Tree Classification (Digits) Accuracy: ",
      accuracy2)

```

F1 score -
confusion

```

# Regression Dataset 1 (Diabetes)
diabetes = load_diabetes()
X, y = diabetes.data[:, :1], diabetes.target

X_train, X_test, y_train, y_test = train_test_split(X, y,
                                                    test_size=0.2, random=42)

reg1 = DecisionTreeRegressor(max_depth=4)
reg1.fit(X_train, y_train)
pred1 = reg1.predict(X_test)
mse1 = math.sqrt(mean_squared_error(y_test, pred1))
print("Decision Tree Regression (Diabetes) RMSE: ", mse1)

```

```

# Regression Dataset 2 (Auto MPG)
url = "https://raw.githubusercontent.com/mwaskom/
      seaborn-data/master/mpg.csv"
data = pd.read_csv(url)
data = data[['horsepower', 'mpg']].dropna()
X, y = data[['horsepower']].values, data['mpg'].values

X_train, X_test, y_train, y_test = train_test_split(X, y,
                                                    test_size=0.2, random_state=42)

reg2 = DecisionTreeRegressor(max_depth=4)
reg2.fit(X_train, y_train)
pred2_reg = reg2.predict(X_test)
rmse2 = math.sqrt(mean_squared_error(y_test,
                                       pred2_reg))

print("Decision Tree Regression (Auto MPG) RMSE: ", rmse2)

```

Output

Decision Tree Classification (Iris) Accuracy: 0.93333

Decision Tree Classification (Digits) Accuracy: 0.53611

Decision Tree Regression (Diabetes) RMSE: 77.39

Decision Tree Regression (Auto MPG) RMSE: 24.1741

Code :

```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
from collections import Counter
from sklearn.datasets import load_iris, load_digits, load_diabetes
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.metrics import accuracy_score, mean_squared_error
import math

# ===== DECISION TREE CLASSIFIER ===== #class
DecisionTreeClassifier:
    def __init__(self, max_depth=5):
        self.max_depth = max_depth

    def fit(self, X, y):
        self.tree = self._build_tree(X, y)

    def _entropy(self, y):
        hist = np.bincount(y)
        ps = hist / len(y)
        return -np.sum([p * np.log2(p) for p in ps if p > 0])

    def _best_split(self, X, y):
        best_gain = -1
        split_idx, split_thresh = None, None
        parent_entropy = self._entropy(y)
        for feature in range(X.shape[1]):
            thresholds = np.unique(X[:, feature])
            for thresh in thresholds:
                left = y[X[:, feature] <= thresh]
                right = y[X[:, feature] > thresh]
                if len(left) == 0 or len(right) == 0:
                    continue
                gain = parent_entropy - (len(left)/len(y))*self._entropy(left) -
(len(right)/len(y))*self._entropy(right)
                if gain > best_gain:
                    best_gain = gain
                    split_idx = feature
                    split_thresh = thresh
        return split_idx, split_thresh

    def _build_tree(self, X, y, depth=0):
        if depth >= self.max_depth or len(set(y)) == 1:
            return Counter(y).most_common(1)[0][0]
        feat, thresh = self._best_split(X, y)
        if feat is None:
            return Counter(y).most_common(1)[0][0]
        left_idx = X[:, feat] <= thresh
        right_idx = X[:, feat] > thresh
        left = self._build_tree(X[left_idx], y[left_idx], depth+1)
        right = self._build_tree(X[right_idx], y[right_idx], depth+1)
```

```

        return (feat, thresh, left, right)

    def _predict(self, x, tree):
        if not isinstance(tree, tuple):
            return tree
        feat, thresh, left, right = tree
        if x[feat] <= thresh:
            return self._predict(x, left)
        else:
            return self._predict(x, right)

    def predict(self, X):
        return np.array([self._predict(x, self.tree) for x in X])

# ===== DECISION TREE REGRESSOR ===== #
class DecisionTreeRegressor:
    def __init__(self, max_depth=5):
        self.max_depth = max_depth

    def fit(self, X, y):
        self.tree = self._build_tree(X, y)

    def _mse(self, y):
        return np.mean((y - np.mean(y))**2)

    def _best_split(self, X, y):
        best_error = float("inf")
        split_idx, split_thresh = None, None
        for feature in range(X.shape[1]):
            thresholds = np.unique(X[:, feature])
            for thresh in thresholds:
                left = y[X[:, feature] <= thresh]
                right = y[X[:, feature] > thresh]
                if len(left) == 0 or len(right) == 0:
                    continue
                error = (len(left)*self._mse(left) + len(right)*self._mse(right)) /
len(y)
                if error < best_error:
                    best_error = error
                    split_idx = feature
                    split_thresh = thresh
        return split_idx, split_thresh

    def _build_tree(self, X, y, depth=0):
        if depth >= self.max_depth:
            return np.mean(y)
        feat, thresh = self._best_split(X, y)
        if feat is None:
            return np.mean(y)
        left_idx = X[:, feat] <= thresh
        right_idx = X[:, feat] > thresh
        left = self._build_tree(X[left_idx], y[left_idx], depth+1)
        right = self._build_tree(X[right_idx], y[right_idx], depth+1)
        return (feat, thresh, left, right)

```

```

def _predict(self, x, tree):
    if not isinstance(tree, tuple):
        return tree
    feat, thresh, left, right = tree
    if x[feat] <= thresh:
        return self._predict(x, left)
    else:
        return self._predict(x, right)

def predict(self, X):
    return np.array([self._predict(x, self.tree) for x in X])

# ===== CLASSIFICATION DATASET 1 (Iris) ===== #
iris = load_iris()
X = iris.data[:, :2]
y = iris.target
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)
clf1 = DecisionTreeClassifier(max_depth=4)
clf1.fit(X_train, y_train)
pred1 = clf1.predict(X_test)
accuracy1 = accuracy_score(y_test, pred1)
print("Decision Tree Classification (Iris) Accuracy:", accuracy1)

# Decision boundary
plt.figure(figsize=(7,5))
x_min, x_max = X[:,0].min()-1, X[:,0].max()+1
y_min, y_max = X[:,1].min()-1, X[:,1].max()+1
xx, yy = np.meshgrid(np.arange(x_min, x_max, 0.05),
                    np.arange(y_min, y_max, 0.05))
grid = np.c_[xx.ravel(), yy.ravel()]
Z = clf1.predict(grid).reshape(xx.shape)
plt.contourf(xx, yy, Z, alpha=0.3, cmap='coolwarm')
plt.scatter(X_test[:,0], X_test[:,1], c=y_test, edgecolor='k')
plt.title("Decision Tree Decision Boundary - Iris")
plt.show()

# ===== CLASSIFICATION DATASET 2 (Digits) ===== #
digits = load_digits()
X = digits.data
y = digits.target

# PCA to reduce to 2D for visualization
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)

X_train, X_test, y_train, y_test = train_test_split(X_pca, y, test_size=0.2,
random_state=42)
clf2 = DecisionTreeClassifier(max_depth=6)
clf2.fit(X_train, y_train)
pred2 = clf2.predict(X_test)
accuracy2 = accuracy_score(y_test, pred2)

```

```

print("Decision Tree Classification (Digits) Accuracy:", accuracy2)

# Decision boundary
plt.figure(figsize=(7,5))
x_min, x_max = X_pca[:,0].min()-1, X_pca[:,0].max()+1
y_min, y_max = X_pca[:,1].min()-1, X_pca[:,1].max()+1
xx, yy = np.meshgrid(np.arange(x_min, x_max, 0.05),
                     np.arange(y_min, y_max, 0.05))
grid = np.c_[xx.ravel(), yy.ravel()]
Z = clf2.predict(grid).reshape(xx.shape)
plt.contourf(xx, yy, Z, alpha=0.3, cmap='coolwarm')
plt.scatter(X_test[:,0], X_test[:,1], c=y_test, edgecolor='k')
plt.title("Decision Tree Decision Boundary - Digits (PCA 2D)")
plt.show()

# ===== REGRESSION DATASET 1 (Diabetes) ===== #
diabetes = load_diabetes()
X, y = diabetes.data[:, :1], diabetes.target # use first feature for visualization
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)
reg1 = DecisionTreeRegressor(max_depth=4)
reg1.fit(X_train, y_train)
pred1_reg = reg1.predict(X_test)
rmse1 = math.sqrt(mean_squared_error(y_test, pred1_reg))
print("Decision Tree Regression (Diabetes) RMSE:", rmse1)

# Regression plot
plt.figure(figsize=(7,5))
sorted_idx = np.argsort(X_test[:,0])
plt.scatter(X_test[sorted_idx], y_test[sorted_idx], label="Actual")
plt.plot(X_test[sorted_idx], pred1_reg[sorted_idx], color='red', linewidth=2,
label="Predicted")
plt.xlabel("Feature 1")
plt.ylabel("Target")
plt.title("Decision Tree Regression - Diabetes")
plt.legend()
plt.show()

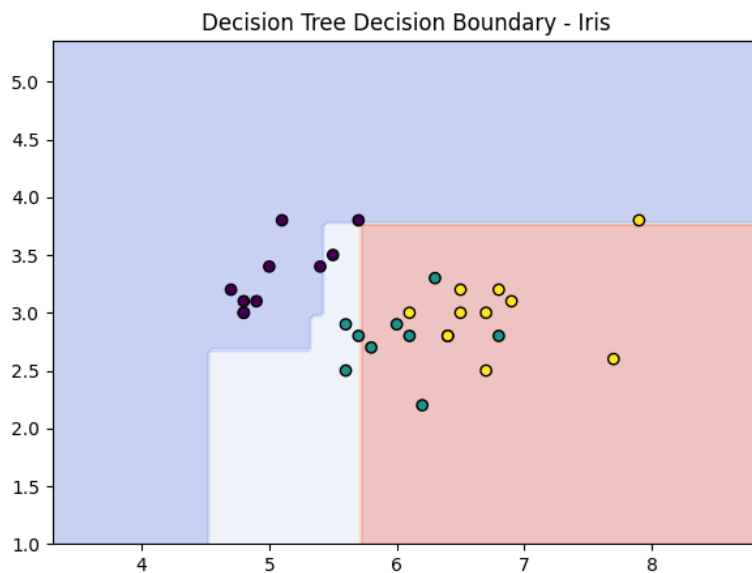
# ===== REGRESSION DATASET 2 (Auto MPG) ===== #
url = "https://raw.githubusercontent.com/mwaskom/seaborn-data/master/mpg.csv"
data = pd.read_csv(url)
data = data[['horsepower', 'mpg']].dropna()
X, y = data[['horsepower']].values, data['mpg'].values
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)
reg2 = DecisionTreeRegressor(max_depth=4)
reg2.fit(X_train, y_train)
pred2_reg = reg2.predict(X_test)
rmse2 = math.sqrt(mean_squared_error(y_test, pred2_reg))
print("Decision Tree Regression (Auto MPG) RMSE:", rmse2)

# Regression plot
plt.figure(figsize=(7,5))
sorted_idx = np.argsort(X_test[:,0])
plt.scatter(X_test[sorted_idx], y_test[sorted_idx], label="Actual")

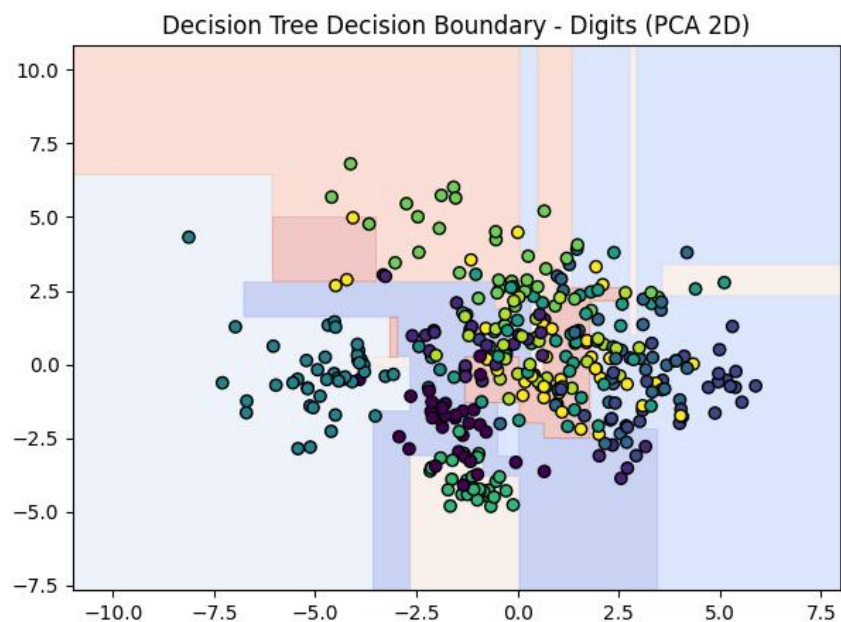
```

```
plt.plot(X_test[sorted_idx], pred2_reg[sorted_idx], color='red', linewidth=2,
label="Predicted")
plt.xlabel("Horsepower")
plt.ylabel("MPG")
plt.title("Decision Tree Regression - Auto MPG")
plt.legend()
plt.show()
```

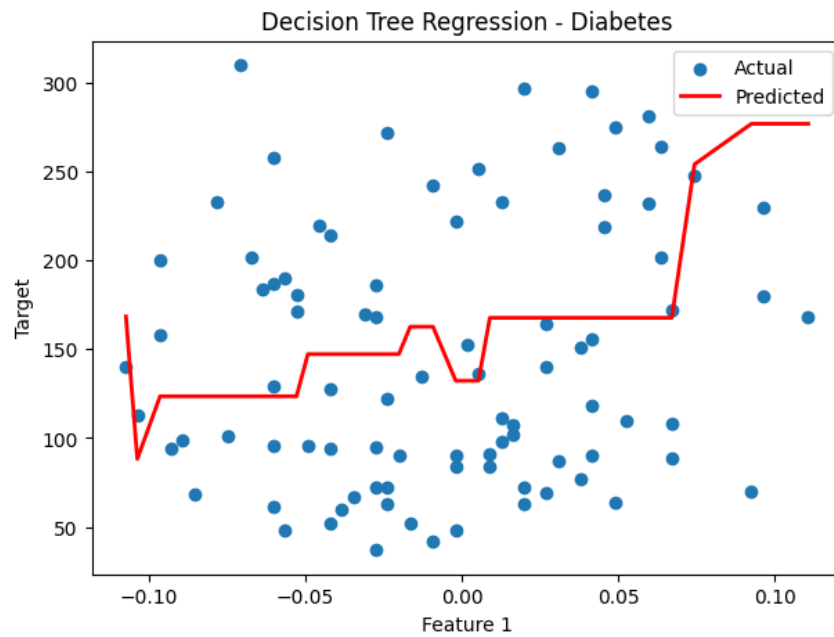
Decision Tree Classification (Iris) Accuracy: 0.7333333333333333



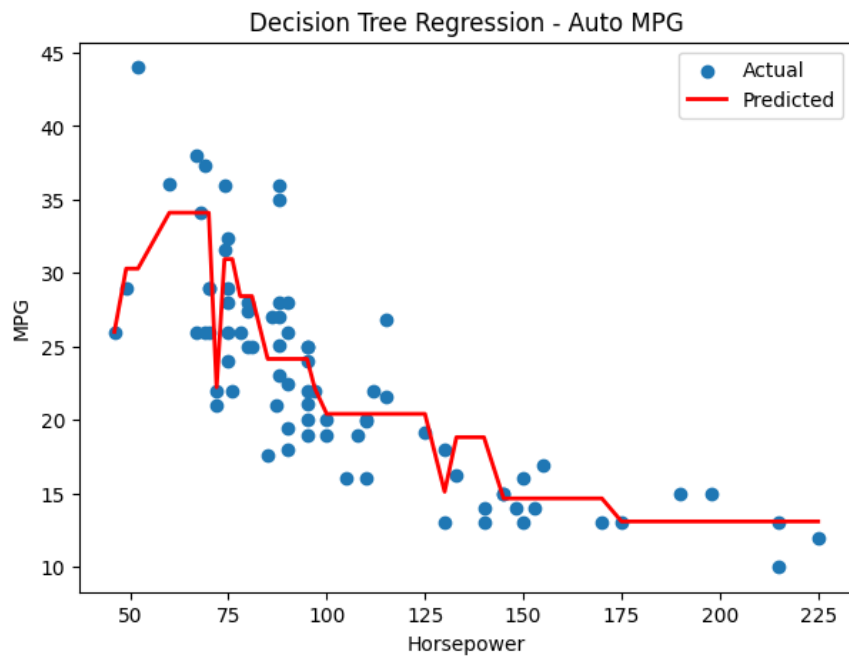
Decision Tree Classification (Digits) Accuracy: 0.5361111111111111



Decision Tree Regression (Diabetes) RMSE: 77.93938938203608



Decision Tree Regression (Auto MPG) RMSE: 4.174151730558545



Program-6

Build Support vector machine model for a given dataset

Observation :

```
Lab-5 7-3-2020

import numpy as np
import matplotlib.pyplot as plt
from sklearn import datasets
from sklearn.svm import SVC
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score,
classification_report, confusion_matrix
import seaborn as sns

iris = datasets.load_iris()

X = iris.data[iris.target != 2, :2]
y = iris.target[iris.target != 2]

feature_names = iris.feature_names[:2]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

model = SVC(kernel='linear', C=1)
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print(f"Accuracy: {accuracy_score(y_test, y_pred)*100.0}%")

target_names = iris.target_names[:2]

print("\n Classification Report :\n")
print(classification_report(y_test, y_pred, target_names
                           = target_names))

output
Accuracy: 100.0%

Classification Report
```

	precision	recall	f1-score	support
setosa	1.00	1.00	1.00	12
versicolour	1.00	1.00	1.00	8
accuracy	1.00			
macro avg	1.00	1.00	1.00	20
weighted avg	1.00	1.00	1.00	20

Code :

```
# =====
#   SVM Model – Iris Dataset (Binary Class)
#   With Hyperplane Visualization
# =====

import numpy as np
import matplotlib.pyplot as plt
from sklearn import datasets
from sklearn.svm import SVC
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
import seaborn as sns

# -----
# 1. Load Dataset
# -----
iris = datasets.load_iris()

# Use only 2 classes for binary classification
X = iris.data[iris.target != 2, :2] # first 2 features
y = iris.target[iris.target != 2]

# Actual feature names
feature_names = iris.feature_names[:2]

# -----
# 2. Train Test Split
# -----
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# -----
# 3. Feature Scaling
# -----
scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# -----
# 4. Train SVM Model
# -----
model = SVC(kernel='linear', C=1)
model.fit(X_train, y_train)

# -----
# 5. Predictions (Updated with Names)
# -----
y_pred = model.predict(X_test)
```

```

print(f"Accuracy: {accuracy_score(y_test, y_pred)*100:.2f}%")

target_names = iris.target_names[:2]

print("\nClassification Report:\n")
print(classification_report(y_test, y_pred, target_names=target_names))

# -----
# 6. Confusion Matrix
# -----plt.figure(figsize=(5,4))sns.heatmap(
    confusion_matrix(y_test, y_pred),
    annot=True,
    cmap="Blues",
    fmt="d",
    xticklabels=["Setosa", "Versicolor"],
    yticklabels=["Setosa", "Versicolor"]
)

plt.title("Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()

# -----
# 7. Visualization of Hyperplane & Regions
# -----

def plot_decision_boundary(model, X, y):

    plt.figure(figsize=(8,6))

    # Plot data points
    plt.scatter(
        X[:,0], X[:,1],
        c=y,
        cmap=plt.cm.coolwarm,
        edgecolors='k'
    )

    ax = plt.gca()

    # create grid
    xlim = ax.get_xlim()
    ylim = ax.get_ylim()

    xx = np.linspace(xlim[0], xlim[1], 30)
    yy = np.linspace(ylim[0], ylim[1], 30)

    YY, XX = np.meshgrid(yy, xx)

    xy = np.vstack([XX.ravel(), YY.ravel()]).T

```

```

Z = model.decision_function(xy)
Z = Z.reshape(XX.shape)

# Plot decision boundary
ax.contour(
    XX, YY, Z,
    colors='k',
    levels=[-1, 0, 1],
    alpha=0.7,
    linestyles=['--', '-', '--']
)

# Plot support vectors
ax.scatter(
    model.support_vectors_[0,0],
    model.support_vectors_[0,1],
    s=100,
    linewidth=1,
    facecolors='none',
    edgecolors='black'
)

plt.title("SVM Hyperplane & Decision Boundary")
plt.xlabel(feature_names[0])
plt.ylabel(feature_names[1])
plt.show()

# Plot boundary
plot_decision_boundary(model, X_train, y_train)

# -----
# 8. Decision Region Visualization
# -----

def plot_regions(X, y, model):

    h = 0.02

    x_min, x_max = X[:,0].min() - 1, X[:,0].max() + 1
    y_min, y_max = X[:,1].min() - 1, X[:,1].max() + 1

    xx, yy = np.meshgrid(
        np.arange(x_min, x_max, h),
        np.arange(y_min, y_max, h)
    )

    Z = model.predict(np.c_[xx.ravel(), yy.ravel()])
    Z = Z.reshape(xx.shape)

    plt.figure(figsize=(8,6))

    plt.contourf(xx, yy, Z, cmap=plt.cm.coolwarm, alpha=0.3)

```

```
plt.scatter(
    X[:,0], X[:,1],
    c=y,
    cmap=plt.cm.coolwarm,
    edgecolors='k'
)

plt.title("Binary Classification Decision Regions")
plt.xlabel(feature_names[0])
plt.ylabel(feature_names[1])

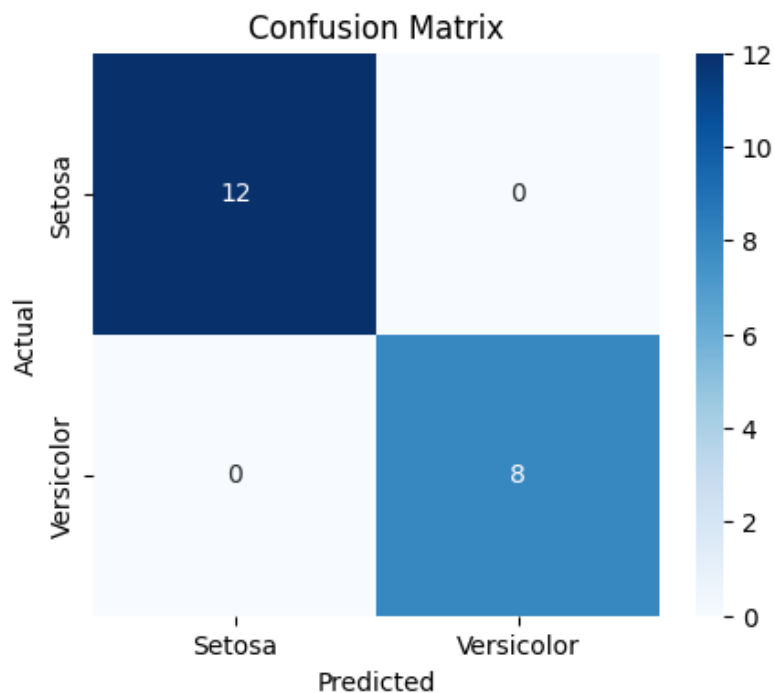
plt.show()

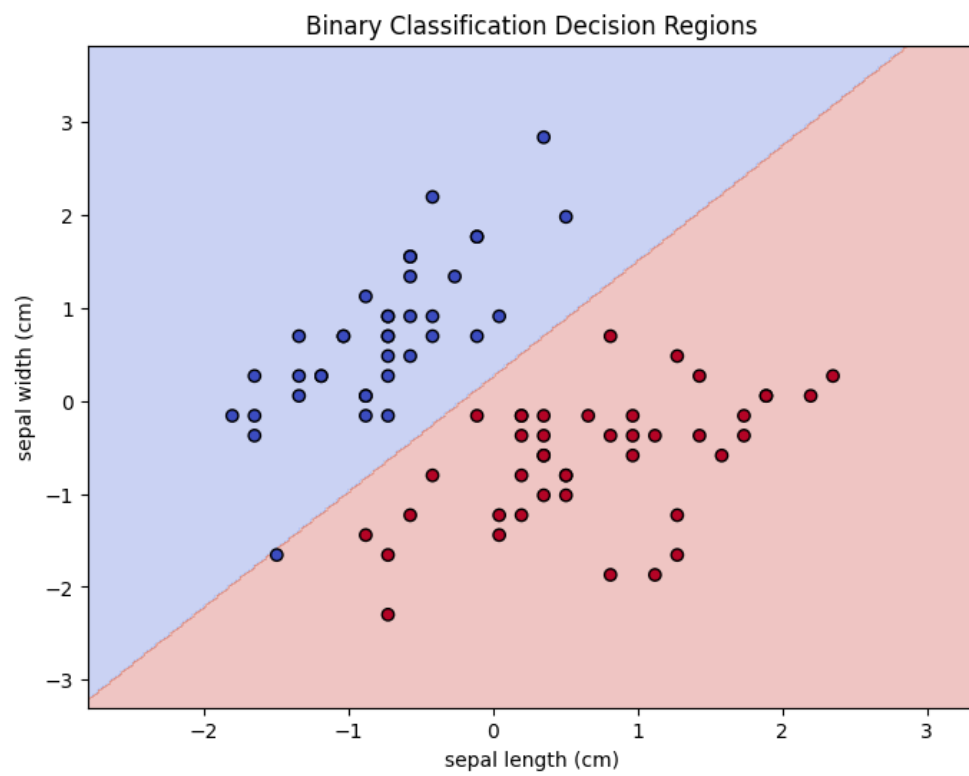
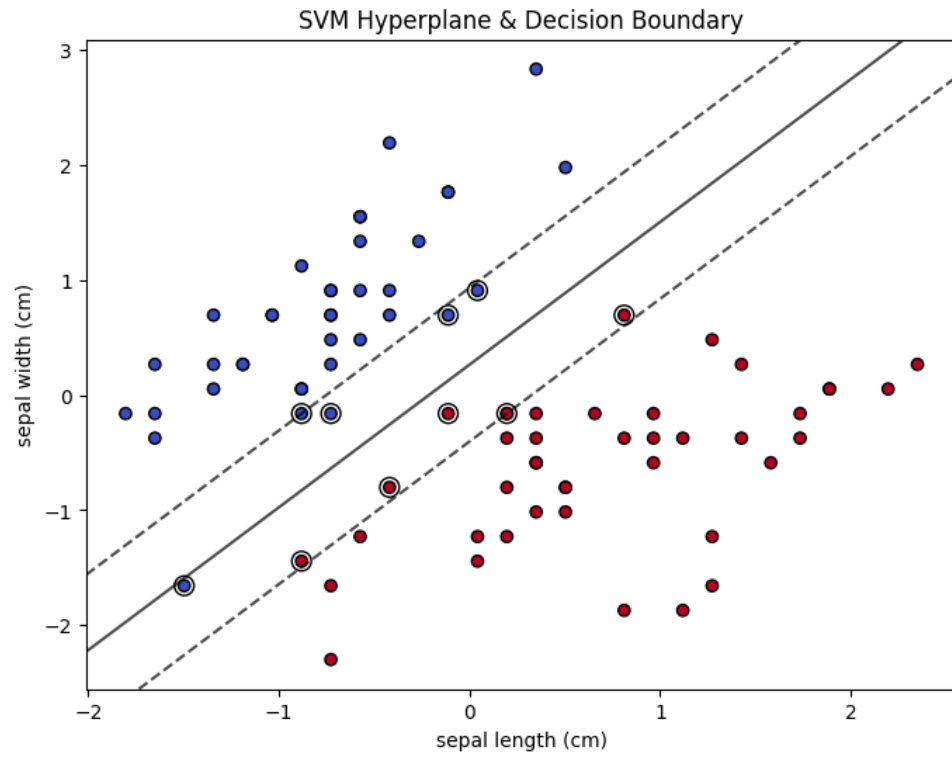
plot_regions(X_train, y_train, model)
```

Accuracy: 100.00%

Classification Report:

	precision	recall	f1-score	support
setosa	1.00	1.00	1.00	12
versicolor	1.00	1.00	1.00	8
accuracy			1.00	20
macro avg	1.00	1.00	1.00	20
weighted avg	1.00	1.00	1.00	20

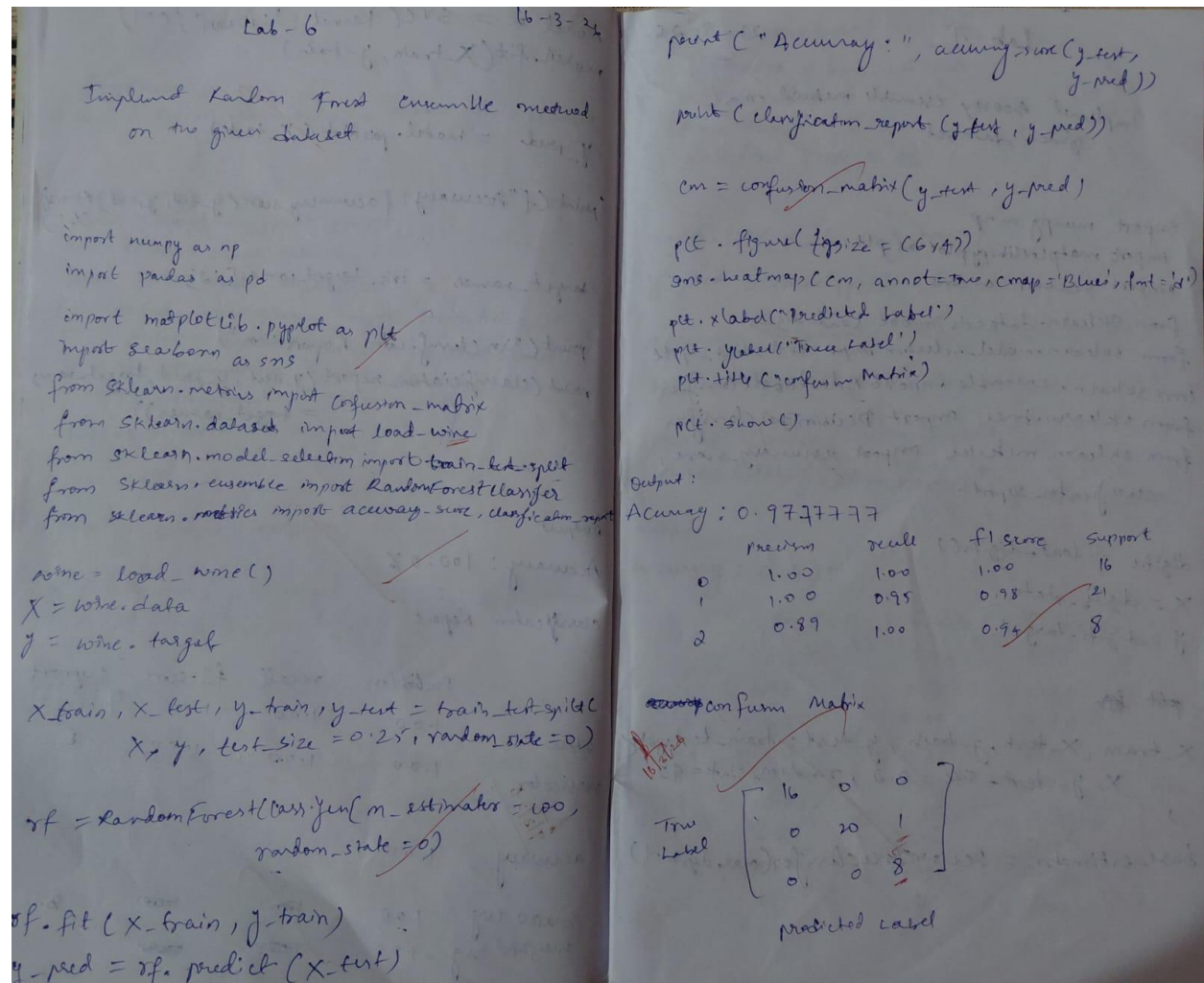




Program-7

Implement Random forest ensemble method on a given dataset.

Observation :



Code :

```
import numpy as np
import pandas as pd

import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.metrics import confusion_matrix

from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report

# Load dataset
wine = load_wine()
X = wine.data
y = wine.target

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=0
)

# Random Forest model
rf = RandomForestClassifier(n_estimators=100, random_state=0)

# Train model
rf.fit(X_train, y_train)

# Prediction
y_pred = rf.predict(X_test)

# Evaluation
print("Accuracy:", accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred))

# Confusion matrix
cm = confusion_matrix(y_test, y_pred)

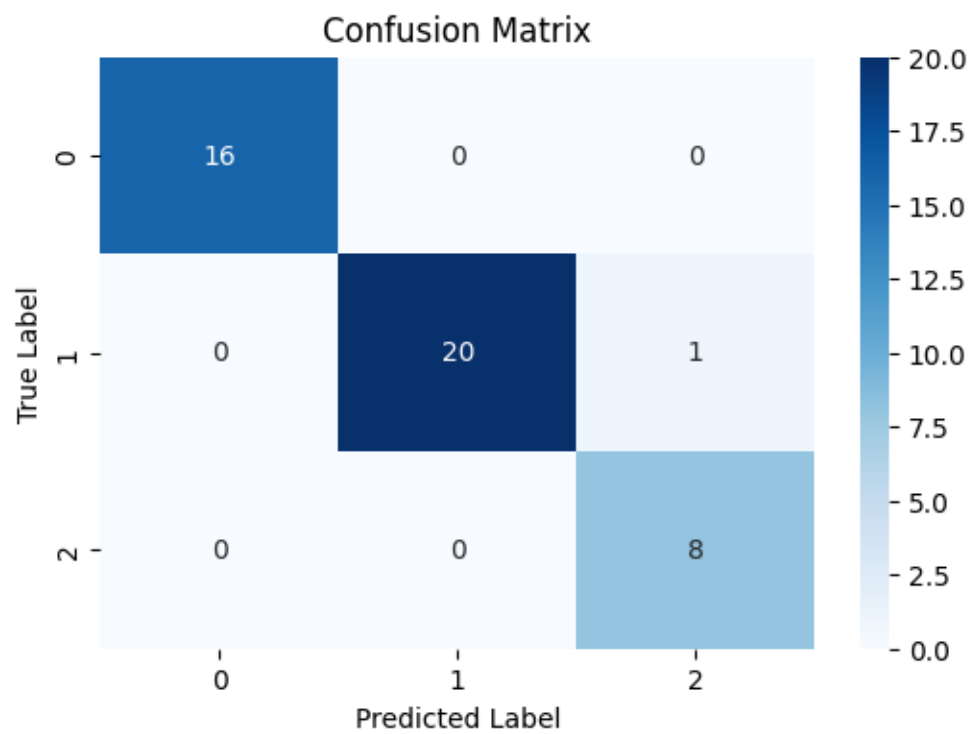
plt.figure(figsize=(6,4))
sns.heatmap(cm, annot=True, cmap="Blues", fmt="d")

plt.xlabel("Predicted Label")
plt.ylabel("True Label")
plt.title("Confusion Matrix")

plt.show()
```

Accuracy: 0.9777777777777777

	precision	recall	f1-score	support
0	1.00	1.00	1.00	16
1	1.00	0.95	0.98	21
2	0.89	1.00	0.94	8
accuracy			0.98	45
macro avg	0.96	0.98	0.97	45
weighted avg	0.98	0.98	0.98	45



Program-8

Implement Boosting ensemble method on a given dataset

Observation :

```
Lab-7 23-3-22
Implement Boosting ensemble method on a
given dataset

import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.ensemble import AdaBoostClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score,
classification_report

digits = load_digits()
X = digits.data
y = digits.target

plt.

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

base_estimator = DecisionTreeClassifier(max_depth=1)

model = AdaBoostClassifier(
    estimator = base_estimator,
    n_estimators = 100,
    learning_rate = 0.5,
    random_state = 42
)

model.fit(X_train, y_train)
y_pred = model.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)
print("Accuracy : ", accuracy)

print("\n Classification Report :\n")
print(classification_report(y_test, y_pred))

output
Accuracy : 0.825
```

	precision	recall	f1-score
0	0.77	0.94	0.95
1	0.57	0.75	0.65
2	0.88	0.67	0.74
3	0.88	0.82	0.85
4	0.84	0.83	0.84
5	0.93	0.85	0.89
6	0.87	0.94	0.92
7	0.89	0.91	0.90
avg	0.57	0.73	0.66
macro	0.84	0.80	0.82

Code :

```
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.ensemble import AdaBoostClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

digits = load_digits()
X = digits.data
y = digits.target

plt.figure(figsize=(8, 4))
for i in range(10):
    plt.subplot(2, 5, i+1)
    plt.imshow(digits.images[i], cmap='gray')
    plt.title(f"Label: {digits.target[i]}")
    plt.axis('off')
plt.suptitle("Sample Digit Images")
plt.show()

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

base_estimator = DecisionTreeClassifier(max_depth=1)

# AdaBoost model
model = AdaBoostClassifier(
    estimator=base_estimator,
    n_estimators=100,
    learning_rate=0.5,
    random_state=42
)

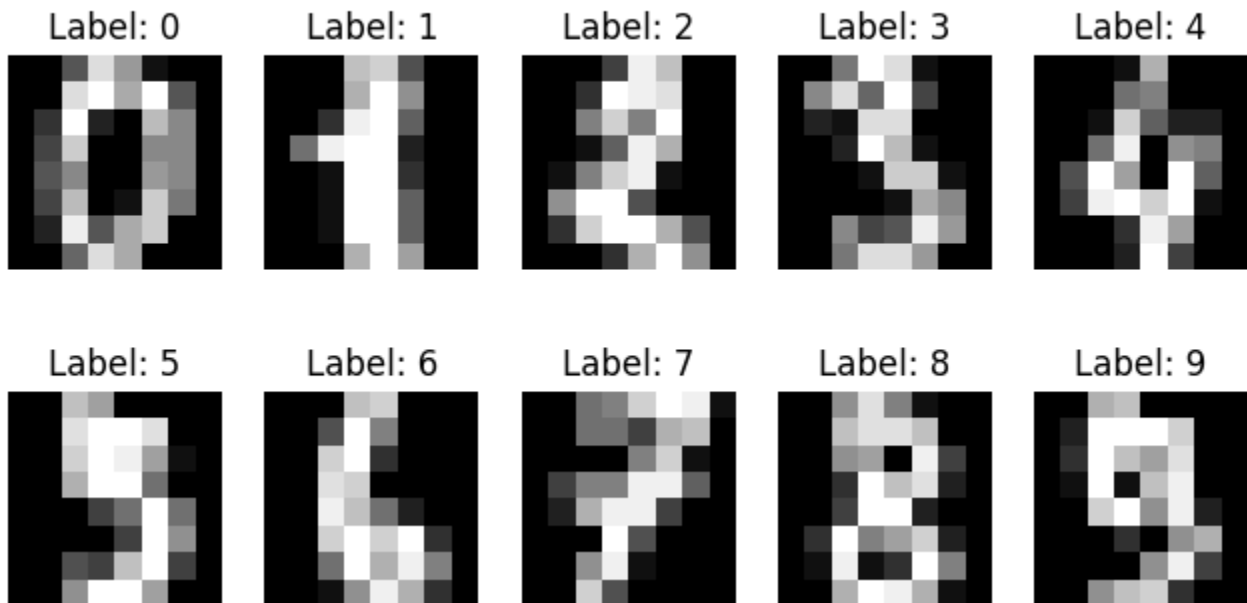
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)
```

```
print("\nClassification Report:\n")
print(classification_report(y_test, y_pred))
```

Sample Digit Images



Accuracy: 0.825

Classification Report:

	precision	recall	f1-score	support
0	0.97	0.94	0.95	33
1	0.57	0.75	0.65	28
2	0.88	0.64	0.74	33
3	0.88	0.82	0.85	34
4	0.84	0.83	0.84	46
5	0.93	0.85	0.89	47
6	0.89	0.94	0.92	35
7	0.89	0.91	0.90	34
8	0.59	0.73	0.66	30
9	0.84	0.80	0.82	40
accuracy			0.82	360
macro avg	0.83	0.82	0.82	360
weighted avg	0.84	0.82	0.83	360

Program-9

Build k-Means algorithm to cluster a set of data stored in a .CSV file

Observation :

Lab-8 17-04-20

Build k-means algorithm to cluster a set of data stored in a .csv file.
clustering mall customer based on spend
dataset :- Mall_Customers.csv

```

import pandas as pd
import matplotlib.pyplot as plt
import sklearn.cluster as cluster
import sklearn.preprocessing as preprocessing

df = pd.read_csv("Mall_Customers.csv")
X = df[["Annual Income (K$)", "Spending Score (1-100)"]]
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

kmeans = KMeans(n_clusters=5, random_state=42)
clusters = kmeans.fit_predict(X_scaled)
df["cluster"] = clusters

centroids = scaler.inverse_transform(kmeans.cluster_centers_)
print("Cluster centers (original scale):", centroids)
df["segment"] = df["cluster"].map(clustername)
print(df.head())
        
```

Output

Cluster centers (original scale):

```

[[55.29, 49.51],
 [86.53, 82.12],
 [25.72, 79.36],
 [88.2, 17.14],
 [26.304, 20.915]]
        
```

CustomerID	Gender	Age	Annual Income (K\$)	Spending Score (1-100)
0	male	19	15	39
1	male	21	15	81
2	female	20	16	6
3	female	23	16	77
4	female	31	17	40

cluster	segment
0	Budget
1	Impulsive
2	Budget
3	Impulsive
4	Budget

df *22/4/20*

Code :

```
import os
os.environ["OMP_NUM_THREADS"] = "1"  # Fix MKL warning

import pandas as pd
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler

# Load dataset
df = pd.read_csv("Mall_Customers.csv")

# Select features
X = df[["Annual Income (k$)", "Spending Score (1-100)"]]

# Scale data
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Train k-Means
kmeans = KMeans(n_clusters=5, random_state=42)
clusters = kmeans.fit_predict(X_scaled)

df["Cluster"] = clusters

# Get centroids in original scale
centroids = scaler.inverse_transform(kmeans.cluster_centers_)

# Assign simple meanings
def label(income, score):
    if income > 70 and score > 60:
        return "Premium"
    elif income > 70 and score < 40:
        return "Careful"
    elif income < 40 and score > 60:
        return "Impulsive"
    elif income < 40 and score < 40:
        return "Budget"
    else:
        return "Standard"

cluster_names = {i: label(c[0], c[1]) for i, c in enumerate(centroids)}

# Plot
plt.figure(figsize=(7,5))

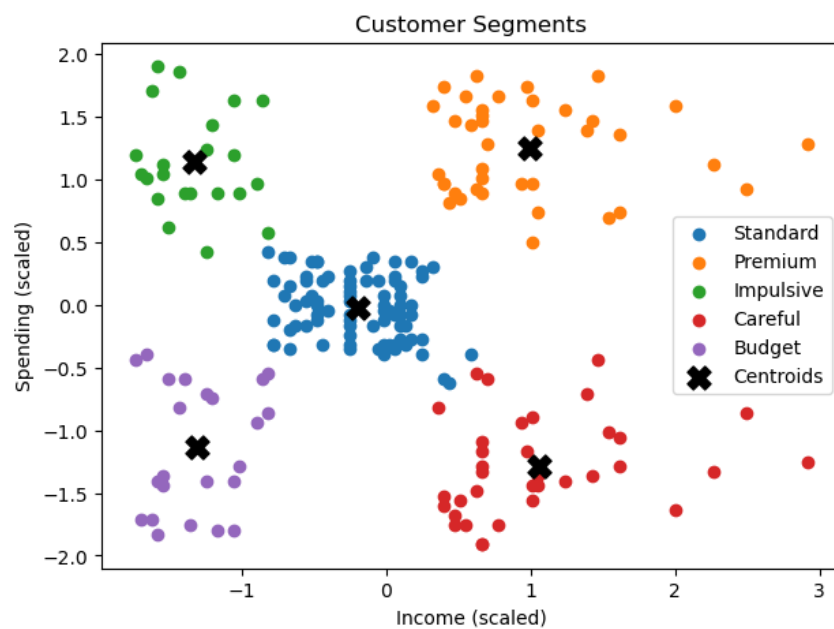
for i in range(5):
    plt.scatter(
        X_scaled[clusters == i, 0],
        X_scaled[clusters == i, 1],
```

```

        label=cluster_names[i]
    )
# Plot centroids
plt.scatter(kmeans.cluster_centers_[ :,0], kmeans.cluster_centers_[ :,1],
            c='black', s=150, marker='X', label='Centroids')
plt.xlabel("Income (scaled)")
plt.ylabel("Spending (scaled)")
plt.title("Customer Segments")
plt.legend()
plt.show()

print("Cluster Centers (original scale):\n", centroids)
df["Segment"] = df["Cluster"].map(cluster_names)
print(df.head())

```



Cluster Centers (original scale):

```

[[55.2962963  49.51851852]
 [86.53846154  82.12820513]
 [25.72727273  79.36363636]
 [88.2         17.11428571]
 [26.30434783  20.91304348]]

```

	CustomerID	Gender	Age	Annual Income (k\$)	Spending Score (1-100)	\
0	1	Male	19	15	39	
1	2	Male	21	15	81	
2	3	Female	20	16	6	
3	4	Female	23	16	77	
4	5	Female	31	17	40	

Cluster	Segment
0	4 Budget
1	2 Impulsive
2	4 Budget
3	2 Impulsive
4	4 Budget

Program-10

Implement Dimensionality reduction using Principle Component Analysis (PCA) method.

Observation :

Lab - 9 17-04-20

Implement Dimensionality reduction using PCA
dataset: Diabetes

```
import matplotlib.pyplot as plt
from sklearn.datasets import load_diabetes
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA

data = load_diabetes()
X = data.data
y = data.target

X_scaled = StandardScaler().fit_transform(X)

pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)

#variance
print("Explained variance:", pca.explained_variance_ratio_)

Output,
Explained variance: [0.40242, 0.14923]
```

22/4/20

Code :

```
import matplotlib.pyplot as plt
from sklearn.datasets import load_diabetes
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
# Load dataset
data = load_diabetes()
X = data.data
y = data.target # disease progression (continuous)

# Scale
X_scaled = StandardScaler().fit_transform(X)

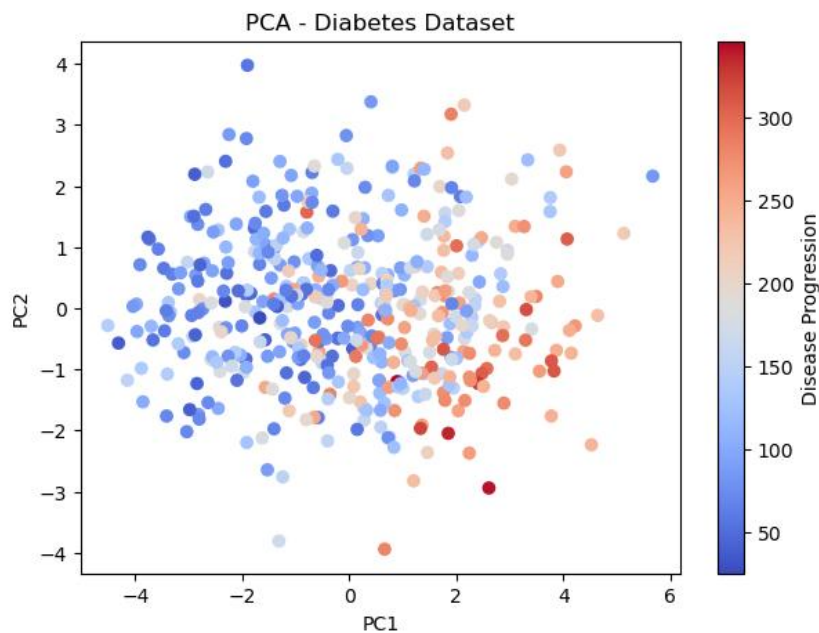
# PCA → 2D
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)

# Plot (color = disease severity)
plt.figure(figsize=(7,5))
sc = plt.scatter(X_pca[:,0], X_pca[:,1], c=y, cmap='coolwarm')

plt.xlabel("PC1")
plt.ylabel("PC2")
plt.title("PCA - Diabetes Dataset")

plt.colorbar(sc, label="Disease Progression")
plt.show()

# Variance
print("Explained Variance:", pca.explained_variance_ratio_)
```



Explained Variance: [0.40242108 0.14923197]

Program-11

Build Logistic Regression Model for a given dataset

Observation :

```
Lab-10  
Implement Logistic Regression  
dataset: titanic_survival.csv  
  
import pandas as pd  
import matplotlib.pyplot as plt  
import seaborn as sns  
from sklearn.model_selection import train_test_split  
from sklearn.preprocessing import LabelEncoder, StandardScaler  
from sklearn.linear_model import LogisticRegression  
from sklearn.metrics import accuracy_score, confusion_matrix  
  
df = pd.read_csv("titanic_survival.csv")  
df = df[["survived", "Pclass", "sex", "Age", "SibSp", "ParCh",  
        "Fare", "Embarked"]]  
df["Age"] = df["Age"].fillna(df["Age"].median())  
df["Embarked"] = df["Embarked"].fillna(df["Embarked"].mode()[0])  
df["sex"] = LabelEncoder().fit_transform(df["sex"])  
df["Embarked"] = LabelEncoder().fit_transform(df["Embarked"])  
X = df.drop("survived", axis=1)  
y = df["survived"]  
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

```

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

model = LogisticRegression(max_iter=300)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))
print("Confusion matrix: \n", confusion_matrix(y_test,
y_pred))

```

Output

Accuracy : 0.804469

Confusion matrix : $\begin{bmatrix} 10 & 15 \\ 20 & 54 \end{bmatrix}$

Code :

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix

df = pd.read_csv("titanic_survival.csv")

df = df[["Survived", "Pclass", "Sex", "Age", "SibSp", "Parch", "Fare", "Embarked"]]

df["Age"] = df["Age"].fillna(df["Age"].median())
df["Embarked"] = df["Embarked"].fillna(df["Embarked"].mode()[0])

df["Sex"] = LabelEncoder().fit_transform(df["Sex"])
df["Embarked"] = LabelEncoder().fit_transform(df["Embarked"])

X = df.drop("Survived", axis=1)
y = df["Survived"]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

model = LogisticRegression(max_iter=300)
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))

plt.figure(figsize=(5,4))
sns.countplot(x=df["Survived"])
plt.title("Survival Count")
plt.show()

cm = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(5,4))
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues")
plt.title("Confusion Matrix")
plt.show()

coeff = model.coef_[0]
plt.figure(figsize=(7,4))
sns.barplot(x=coeff, y=X.columns)
plt.title("Feature Importance")
```

```
plt.show()
```

Accuracy: 0.8044692737430168

Confusion Matrix:

```
[[90 15]
```

```
[20 54]]
```

